# 文档

1. docker官方文档：https://docs.docker.com/network/

# 素材准备

## 包含系列网络工具的镜像

1. dockerfile 定义

```
From alpine:3.18
run apk add ethtool
run apk add ipvsadm
run apk add iptables
```

2. 镜像构建

```
docker build -t myalpine .
```

# 网络解决什么问题

1. 容器与外界通信
2. 容器间通讯，跨主机容器间通讯
3. 网络隔离（容器网络命名空间、子网隔离）
4. 提供网络自定义能力
5. 提供容器间发现功能
6. 提供负载均衡能力

# docker网络命令

```
//连接一个容器到一个网络
docker network connect     Connect a container to a network
//创建一个网络
docker network create      Create a network
//将容器从一个网络中断开
docker network disconnect  Disconnect a container from a network
//查看网络的详细信息
docker network inspect     Display detailed information on one or more networks
//查看网络列表
docker network ls          List networks
//移除所有未使用的网络
docker network prune       Remove all unused networks
//移除一个或多个网络
docker network rm          Remove one or more networks
```

# 常见网络操作

## 发布端口

1. 发布端口

```
# 将容器中tcp端口80映射到主机8080端口
-p 8080:80
# 将容器中tcp端口80映射到地址192.168.239.154:8080
-p 192.168.239.154:8080:80
# 将容器中的udp端口80映射到主机8080端口
-p 8080:80/udp
# 将容器中的udp端口80映射到主机的udp端口8080
# 将容器中的tcp端口80映射到主机的tcp端口8080
-p 8080:80/tcp -p 8080:80/udp
```

2. 案例

```
docker run -d --rm --name mynginx -p 8080:80 nginx:1.23.4
curl http://localhost:8080
```

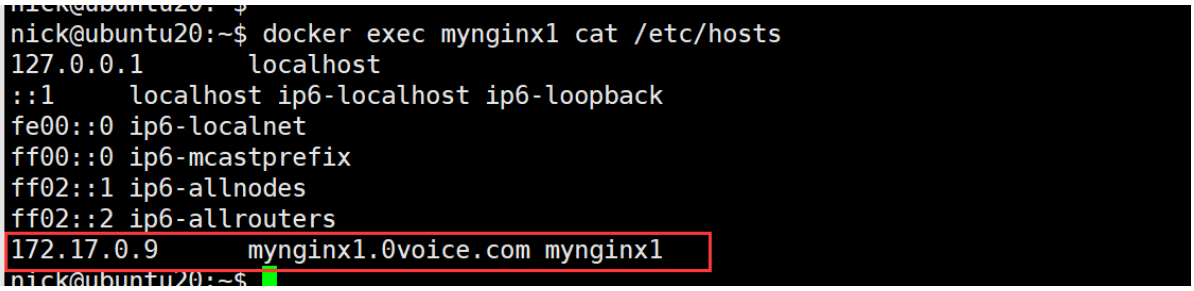
## 修改hostname

1. --hostname 选项可以修改容器/etc/hosts文件下的映射

```
docker run -d --rm --name mynginx1 -p 8081:80 --hostname mynginx1.0voice.com
nginx:1.23.4
curl http://localhost:8081
```

2. 查看容器的/etc/hosts文件

```
docker exec mynginx1 cat /etc/hosts
```




3. 与容器交互下，在容器环境中访问hostname

```
nick@ubuntu20:~$ docker exec -it mynginx1 bash
root@mynginx1:/# curl http://mynginx1.0voice.com
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@mynginx1:/#
```

# 网络驱动

## bridge（网桥网络）

### 什么是网桥

1. bridge是docker默认的网络驱动程序。如果没有指定驱动程序，这就是正在创建的网络类型。当应用程序在需要与同一主机上的其他容器通信的容器中运行时，通常会使用桥接网络。
2. 就联网而言，桥接网络是在网段之间转发流量的链路层设备，链路层设备Mac地址进行通信。桥接器可以是在主机内核运行的硬件设备或者软件设备。
3. 就docker而言，网桥网络为软件网桥。允许连接到同一网桥的容器进行通讯，同时提供与未连接到该网桥的容器的隔离。不同网桥上的容器无法直接通讯。
4. 启动docker是，会自动创建一个名为docker0的网桥网络，并且新启动的容器会默认连接到该网络。

### 用户自定义网桥与默认网桥的区别

1. 自定义网桥提供容器之间的自动DNS解析，可通过容器名称或别名互相访问，默认网桥网络上的容器只能通过IP地址互相访问。
2. 自定义网桥提供更好的网络隔离，因为所有未指定网络的容器都将连接到默认网桥，而自定义网桥则必须显示指定容器网络后，方可加入该网络。
3. 每个自定义网络都会创建一个可配置的网桥。如果使用默认网桥，对其配置将使得所有容器都使用相同的设置，此外配置默认网桥发生在docker本身之外，需要重新启动docker。使用docker网络创建和配置自定义网桥网络，可以在创建是分别对不同的网桥做不同的配置。

### 使用网桥驱动创建网络时传递给--option的特定选项

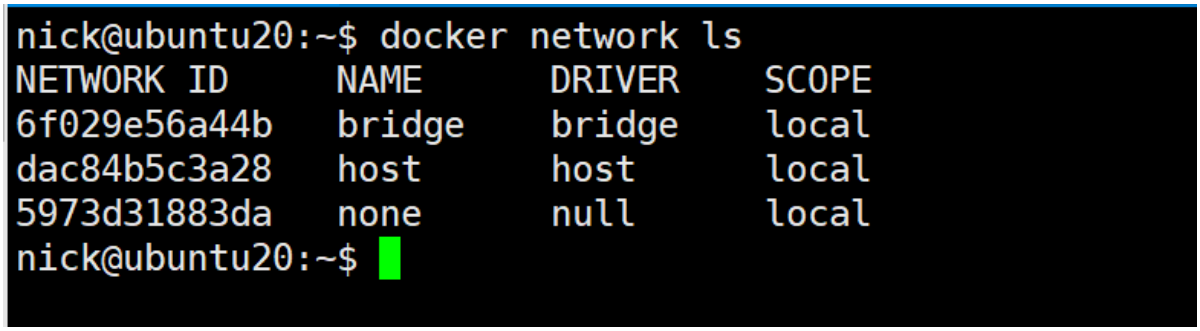
| 选项 | 默认值 | 描述 |
| --- | --- | --- |
| com.docker.network.bridge.name | | 创建Linux网桥时要使用的接口名称 |

| 选项 | 默认值 | 描述 |
| --- | --- | --- |
| com.docker.network.bridge.enable_ip_masquerade | true | 是否启用IP伪装，即将容器的IP地址转换成宿主机的ip地址进行网络访问，从而实现容器访问外部网络的功能，即容器对外部网络访问的权限等同宿主机 |
| com.docker.network.bridge.enable_icc | true | 启用或禁用容器间连接（不是dns解析） |
| com.docker.network.bridge.host_binding_ipv4 | 0.0.0.0 | 绑定容器端口时的默认IP |
| com.docker.network.driver.mtu | 0(不限制) | 设置容器最大传输单元 |
| com.docker.network.container_iface_prefix | eth | 为容器接口设置自定义前缀 |

## 查看默认网络

1. 查看网络

```
docker network ls
```




安装docker后，会默认创建bridge、host、none 三个网络

2. 默认桥接网络

```
nick@ubuntu20:~$ docker network inspect bridge
[
    {
        "Name": "bridge",
        "Id": "6f029e56a44b9ce39be752615ac1b1318277c1529525119657340a8e5bc3785e",
        "Created": "2023-07-26T16:07:41.637589212Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "172.17.0.0/16",
                    "Gateway": "172.17.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {},
        "Options": {
            "com.docker.network.bridge.default_bridge": "true",
            "com.docker.network.bridge.enable_icc": "true",
            "com.docker.network.bridge.enable_ip_masquerade": "true",
            "com.docker.network.bridge.host_binding_ipv4": "0.0.0.0",
            "com.docker.network.bridge.name": "docker0",
            "com.docker.network.driver.mtu": "1500"
        },
        "Labels": {}
    }
]
```

默认桥接网络子网为：172.17.0.0/16，网桥名称为docker0

3. 查看本机网卡信息

```
nick@ubuntu20:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:2e:8d:2c brd ff:ff:ff:ff:ff:ff
    inet 192.168.239.154/24 brd 192.168.239.255 scope global dynamic ens33
       valid_lft 1234sec preferred_lft 1234sec
    inet6 fe80::20c:29ff:fe2e:8d2c/64 scope link
       valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:65:44:be:65 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
    inet6 fe80::42:65ff:fe44:be65/64 scope link
       valid_lft forever preferred_lft forever
```

4. linux内核路由表

```
nick@ubuntu20:~$ route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0         192.168.239.2   0.0.0.0         UG    100    0        0 ens33
172.17.0.0      0.0.0.0         255.255.0.0     U     0      0        0 docker0
192.168.239.0   0.0.0.0         255.255.255.0   U     0      0        0 ens33
192.168.239.2   0.0.0.0         255.255.255.255 UH    100    0        0 ens33
nick@ubuntu20:~$
```

所有以172.17开头并且子网掩码为255.255.0.0的IP地址的数据包都会被发送到docker0网桥

5. iptables 规则，docker安装后会自动配置iptables规则来管理网络流量

```
nick@ubuntu20:~$ sudo iptables -L
[sudo] password for nick:
Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain FORWARD (policy DROP)
target     prot opt source               destination
DOCKER-USER  all  --  anywhere             anywhere
DOCKER-ISOLATION-STAGE-1  all  --  anywhere             anywhere
ACCEPT     all  --  anywhere             anywhere             ctstate RELATED,ESTABLISHED
DOCKER     all  --  anywhere             anywhere
ACCEPT     all  --  anywhere             anywhere
ACCEPT     all  --  anywhere             anywhere

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination

Chain DOCKER (1 references)
target     prot opt source               destination

Chain DOCKER-ISOLATION-STAGE-1 (1 references)
target     prot opt source               destination
DOCKER-ISOLATION-STAGE-2  all  --  anywhere             anywhere
RETURN     all  --  anywhere             anywhere

Chain DOCKER-ISOLATION-STAGE-2 (1 references)
target     prot opt source               destination
DROP       all  --  anywhere             anywhere
RETURN     all  --  anywhere             anywhere

Chain DOCKER-USER (1 references)
target     prot opt source               destination
RETURN     all  --  anywhere             anywhere
nick@ubuntu20:~$
```

1. Chain INPUT (policy ACCEPT)：Chain INPUT 是一个默认的过滤链，它决定了进入本地系统的数据包如何处理。policy ACCEPT 表示该链的默认策略是接受（allow）所有进入的数据包。
2. Chain OUTPUT (policy ACCEPT)：OUTPUT链用于处理从本地系统发出的数据包。它决定了本地系统发往外部网络的流量如何进行过滤和处理。默认情况下，OUTPUT链的策略是ACCEPT，意味着所有从本地系统发出的数据包都会被允许通过。
3. Chain FORWARD (policy DROP)：FORWARD链用于处理转发（Forwarding）数据包，即通过本地系统进行转发的数据包。它决定了经过本地系统的数据包如何进行过滤和处理。默认情况下，FORWARD链的策略是DROP，意味着所有经过本地系统的转发数据包都会被丢弃。这样可以确保只有经过明确允许的规则才能进行转发
4. Chain DOCKER：DOCKER链用于处理与Docker容器相关的网络流量。它是由Docker自动创建的一个用户定义链（user-defined chain），用于进行NAT(Network Address Translation)和过滤规则。
5. Chain DOCKER-USER：DOCKER-USER链是Docker自动创建的另一个用户定义链（user-defined chain），用于处理与Docker容器相关的用户定义规则。
6. Chain DOCKER-ISOLATION-STAGE：DOCKER-ISOLATION-STAGE链是Docker自动创建的用户定义链（user-defined chain），用于实现容器网络隔离。
7. 查看iptables nat表规则列表

```
nick@ubuntu20:~$ sudo iptables -t nat -vnL
[sudo] password for nick:
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    4   208 DOCKER     all  --  *      *       0.0.0.0/0            0.0.0.0/0            ADDRTYPE match dst-type LOCAL

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 DOCKER     all  --  *      *       0.0.0.0/0           !127.0.0.0/8          ADDRTYPE match dst-type LOCAL

Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 MASQUERADE  all  --  *      !docker0  172.17.0.0/16        0.0.0.0/0

Chain DOCKER (2 references)
 pkts bytes target     prot opt in     out     source               destination
    0     0 RETURN     all  --  docker0 *       0.0.0.0/0            0.0.0.0/0
```

1. -t nat: 指定使用nat表，即网络地址转换表
2. -v: 显示详细信息，包括数据包和字节计数
3. n: 不进行域名解析，直接显示IP地址
4. -L: 列出规则列表
5. Chain PREROUTING：（预路由链）是iptables中的一个网络地址转换表（nat表）的链。它用于在数据包到达主机之前进行处理和转发
6. Chain POSTROUTING：（后路由链）是iptables中的一个网络地址转换表（nat表）的链。它用于在数据包离开主机之前进行处理和转发。
7. 规则解读

```
Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 MASQUERADE  all  --  *      !docker0  172.17.0.0/16        0.0.0.0/0
```

表示对于从不在docker0接口上的网络流量（即输出流量），如果源IP地址属于172.17.0.0/16网段，则进行伪装（MASQUERADE），将源IP地址替换为本机的外部IP地址，以实现网络地址转换（NAT）功能。这样可以确保来自172.17.0.0/16网段的数据包能够正确地回应到发起请求的主机

# bridge驱动使用案例

## 使用默认桥接网络

1. 启动容器

```
docker run -dit --rm --name alpine1 myalpine ash
docker run -dit --rm --name alpine2 myalpine ash
```

```
nick@ubuntu20:~/alpine$ docker ps
CONTAINER ID   IMAGE      COMMAND   CREATED          STATUS          PORTS     NAMES
757158e40ae1   myalpine   "ash"     54 seconds ago   Up 53 seconds             alpine2
a73532fbf578   myalpine   "ash"     55 seconds ago   Up 54 seconds             alpine1
nick@ubuntu20:~/alpine$
```

运行容器，在未指定任何网络的情况下，容器将连接到默认的bridge网络。

2. 查看bridge网络

```
docker network inspect bridge
```

```
nick@ubuntu20:~/alpine$ docker network inspect bridge
[
    {
        "Name": "bridge",
        "Id": "6f029e56a44b9ce39be752615ac1b1318277c1529525119657340a8e5bc3785e",
        "Created": "2023-07-26T16:07:41.637589212Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "172.17.0.0/16",
                    "Gateway": "172.17.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {
            "757158e40ae191becf34010078ea1964784406ec86a8f644e5393e34b6b7505b": {
                "Name": "alpine2",
                "EndpointID": "8b9d652ef152853cc03ab484eb3f9a7192d0ee36b2db28140d4d5a0a40dce07c",
                "MacAddress": "02:42:ac:11:00:03",
                "IPv4Address": "172.17.0.3/16",
                "IPv6Address": ""
            },
            "a73532fbf578d36eba969b5d28001674ec9d2a949a7a44035e62e3c72951dd3a": {
                "Name": "alpine1",
                "EndpointID": "35e064a6a19ad5ec17452f9ad0ea658b3d8752000b478a4da085fb2aee2a555f",
                "MacAddress": "02:42:ac:11:00:02",
                "IPv4Address": "172.17.0.2/16",
                "IPv6Address": ""
            }
        },
        "Options": {
            "com.docker.network.bridge.default_bridge": "true",
            "com.docker.network.bridge.enable_icc": "true",
            "com.docker.network.bridge.enable_ip_masquerade": "true",
            "com.docker.network.bridge.host_binding_ipv4": "0.0.0.0",
            "com.docker.network.bridge.name": "docker0",
            "com.docker.network.driver.mtu": "1500"
        },
        "Labels": {}
    }
]
```

3. 通过ip addr 查看系统网络接口，多出两个新的接口

```
nick@ubuntu20:~/alpine$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:2e:8d:2c brd ff:ff:ff:ff:ff:ff
    inet 192.168.239.154/24 brd 192.168.239.255 scope global dynamic ens33
       valid_lft 1357sec preferred_lft 1357sec
    inet6 fe80::20c:29ff:fe2e:8d2c/64 scope link
       valid_lft forever preferred_lft forever
3: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:65:44:be:65 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
    inet6 fe80::42:65ff:fe44:be65/64 scope link
       valid_lft forever preferred_lft forever
28: vetheacab4c@if27: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
    link/ether 8a:bc:de:1e:b3:b3 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet6 fe80::88bc:deff:fe1e:b3b3/64 scope link
       valid_lft forever preferred_lft forever
30: veth021361c@if29: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
    link/ether 02:9e:86:37:23:bb brd ff:ff:ff:ff:ff:ff link-netnsid 1
    inet6 fe80::9e:86ff:fe37:23bb/64 scope link
       valid_lft forever preferred_lft forever
nick@ubuntu20:~/alpine$
```

4. brctl show 查看网桥信息（需要安装bridge-utils软件包）

```
nick@ubuntu20:~/alpine$ brctl show
bridge name     bridge id               STP enabled     interfaces
docker0         8000.02426544be65       no              veth021361c
                                                        vetheacab4c

nick@ubuntu20:~/alpine$
```

5. 容器连接到网桥的虚拟设备对（veth pair）

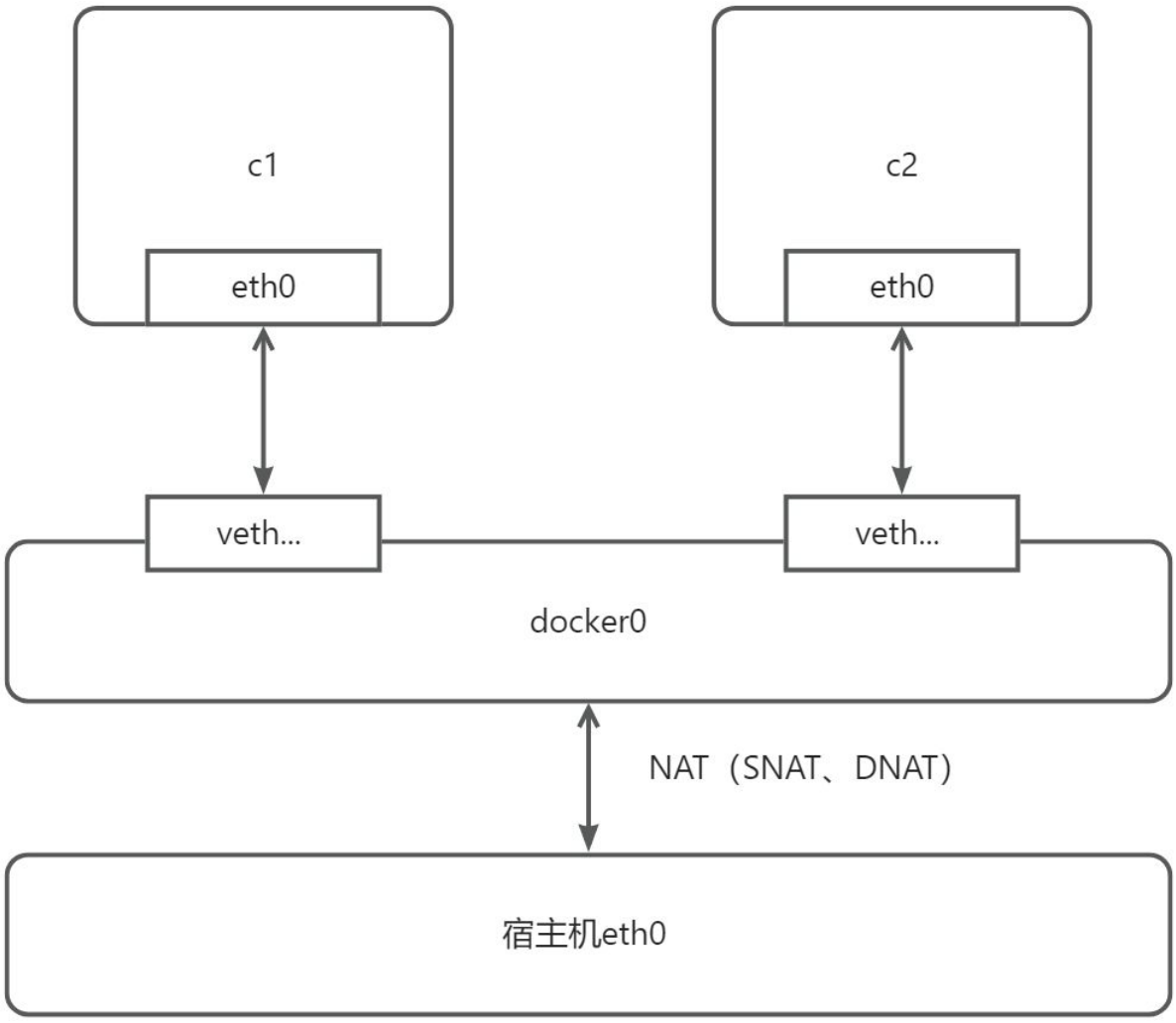
```
28: vetheacab4c@if27: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
    link/ether 8a:bc:de:1e:b3:b3 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet6 fe80::88bc:deff:fe1e:b3b3/64 scope link
       valid_lft forever preferred_lft forever
30: veth021361c@if29: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
    link/ether 02:9e:86:37:23:bb brd ff:ff:ff:ff:ff:ff link-netnsid 1
    inet6 fe80::9e:86ff:fe37:23bb/64 scope link
       valid_lft forever preferred_lft forever
nick@ubuntu20:~$ ethtool -S vetheacab4c
NIC statistics:
     peer_ifindex: 27
      rx_queue_0_xdp_packets: 0
      rx_queue_0_xdp_bytes: 0
      rx_queue_0_xdp_drops: 0
nick@ubuntu20:~$
```

Host主机索引为28的接口，与其配对的接口索引为27

```
nick@ubuntu20:~$ docker exec -it alpine1  ash
/ # ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
27: eth0@if28: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue state UP
    link/ether 02:42:ac:11:00:02 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
       valid_lft forever preferred_lft forever
/ # ethtool -S eth0
NIC statistics:
     peer_ifindex: 28
      rx_queue_0_xdp_packets: 0
      rx_queue_0_xdp_bytes: 0
      rx_queue_0_xdp_drops: 0
/ #
```

容器内部索引为27的接口，与其配对的接口索引为28

6. docker0网桥工作示意图




7. 默认桥接网络不提供dns解析，容器之间可以通过ip地址相互访问，但不能通过容器名称相互访问

1. 与alpine1交互，ping www.0voice.com

```
nick@ubuntu20:~$ docker exec -it alpine1 ash
/ # ping www.0voice.com
PING www.0voice.com (42.194.128.13): 56 data bytes
64 bytes from 42.194.128.13: seq=0 ttl=127 time=18.531 ms
64 bytes from 42.194.128.13: seq=1 ttl=127 time=22.215 ms
64 bytes from 42.194.128.13: seq=2 ttl=127 time=23.706 ms
64 bytes from 42.194.128.13: seq=3 ttl=127 time=21.511 ms
^C
--- www.0voice.com ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 18.531/21.490/23.706 ms
/ #
```

2. 与alpine2交互，ping www.0voice.com

```
nick@ubuntu20:~$ docker exec -it alpine2 ash
/ # ping www.0voice.com
PING www.0voice.com (42.194.128.13): 56 data bytes
64 bytes from 42.194.128.13: seq=0 ttl=127 time=18.740 ms
64 bytes from 42.194.128.13: seq=1 ttl=127 time=22.630 ms
64 bytes from 42.194.128.13: seq=2 ttl=127 time=23.340 ms
64 bytes from 42.194.128.13: seq=3 ttl=127 time=21.622 ms
^C
--- www.0voice.com ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 18.740/21.583/23.340 ms
/ #
```

3. 与alpine2交互，ping alpine1与alpine1对应的ip地址

```
nick@ubuntu20:~$ docker exec -it alpine2 ash
/ # ping alpine1
ping: bad address 'alpine1'
/ # ping 172.17.0.2
PING 172.17.0.2 (172.17.0.2): 56 data bytes
64 bytes from 172.17.0.2: seq=0 ttl=64 time=0.806 ms
64 bytes from 172.17.0.2: seq=1 ttl=64 time=0.125 ms
^C
--- 172.17.0.2 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.125/0.465/0.806 ms
```

## 自定义bridge网络

1. 自定义网络

```
docker network create --driver bridge alpine-net
```

```
nick@ubuntu20:~$ docker network create --driver bridge alpine-net
957175b97b06d11b40a2b62ea5bdbf57166da9da8e4edbd8f924520afae74b79
nick@ubuntu20:~$ docker network inspect alpine-net
[
    {
        "Name": "alpine-net",
        "Id": "957175b97b06d11b40a2b62ea5bdbf57166da9da8e4edbd8f924520afae74b79",
        "Created": "2023-07-27T09:56:56.175354509Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.19.0.0/16",
                    "Gateway": "172.19.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {},
        "Options": {},
        "Labels": {}
    }
]
```

2. 运行两个新的容器，指定网络为自定义的alpine-net

```
docker run -dit --rm --name alpine3 --network alpine-net myalpine ash
docker run -dit --rm --name alpine4 --network alpine-net myalpine ash
```

3. 查看alpine-net网络详情

```
nick@ubuntu20:~$ docker network inspect alpine-net
[
    {
        "Name": "alpine-net",
        "Id": "957175b97b06d11b40a2b62ea5bdbf57166da9da8e4edbd8f924520afae74b79",
        "Created": "2023-07-27T09:56:56.175354509Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.19.0.0/16",
                    "Gateway": "172.19.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {
            "21e125f868cf92a1593622de4956e4635ff067e5102c5acc015329751e26d1f2": {
                "Name": "alpine3",
                "EndpointID": "c0d558a4976fb1210c6516c9a1f27437f7c1b1278629ceb4aa1b25653c360a71",
                "MacAddress": "02:42:ac:13:00:02",
                "IPv4Address": "172.19.0.2/16",
                "IPv6Address": ""
            },
            "b4e40a20edb34d8aae99e4244ad2c7c5d0b3ba8552cc12741b7971398435eda7": {
                "Name": "alpine4",
                "EndpointID": "c7d7651159f5bf203947d6250252c82ccdd8465b8ba1753db82069dcc155f68e",
                "MacAddress": "02:42:ac:13:00:03",
                "IPv4Address": "172.19.0.3/16",
                "IPv6Address": ""
            }
        },
        "Options": {},
        "Labels": {}
    }
]
```

4. 自定义网络容器之间自动dns解析

```
nick@ubuntu20:~$ docker exec -it alpine3 ash
/ # ping alpin4
ping: bad address 'alpin4'
/ # ping alpine4
PING alpine4 (172.19.0.3): 56 data bytes
64 bytes from 172.19.0.3: seq=0 ttl=64 time=0.305 ms
64 bytes from 172.19.0.3: seq=1 ttl=64 time=0.195 ms
64 bytes from 172.19.0.3: seq=2 ttl=64 time=0.136 ms
64 bytes from 172.19.0.3: seq=3 ttl=64 time=0.171 ms
64 bytes from 172.19.0.3: seq=4 ttl=64 time=0.122 ms
^C
--- alpine4 ping statistics ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max = 0.122/0.185/0.305 ms
/ #
```

5. 将已有容器加入到新的网络

```
# 将容器 alpine1 加入网络 alpine-net
docker network connect alpine-net alpine1
# 将容器 alpine2 加入网络 alpine-net
docker network connect alpine-net alpine2
# 查看网络详情
docker network inspect alpine-net
```

```
    "Configonly": false,
    "Containers": {
        "21e125f868cf92a1593622de4956e4635ff067e5102c5acc015329751e26d1f2": {
            "Name": "alpine3",
            "EndpointID": "c0d558a4976fb1210c6516c9a1f27437f7c1b1278629ceb4aa1b25653c360a71",
            "MacAddress": "02:42:ac:13:00:02",
            "IPv4Address": "172.19.0.2/16",
            "IPv6Address": ""
        },
        "757158e40ae191becf34010078ea1964784406ec86a8f644e5393e34b6b7505b": {
            "Name": "alpine2",
            "EndpointID": "b5835db7383eab428a7ea3b211a8ea0155d9047eff715ed42e249c7407372e33",
            "MacAddress": "02:42:ac:13:00:05",
            "IPv4Address": "172.19.0.5/16",
            "IPv6Address": ""
        },
        "a73532fbf578d36eba969b5d28001674ec9d2a949a7a44035e62e3c72951dd3a": {
            "Name": "alpine1",
            "EndpointID": "ac7312cad57a67251997a6794e72dfaad11f0b106bc6e4b3a7fc9edc0863ec70",
            "MacAddress": "02:42:ac:13:00:04",
            "IPv4Address": "172.19.0.4/16",
            "IPv6Address": ""
        },
        "b4e40a20edb34d8aae99e4244ad2c7c5d0b3ba8552cc12741b7971398435eda7": {
            "Name": "alpine4",
            "EndpointID": "c7d7651159f5bf203947d6250252c82ccdd8465b8ba1753db82069dcc155f68e",
            "MacAddress": "02:42:ac:13:00:03",
            "IPv4Address": "172.19.0.3/16",
            "IPv6Address": ""
        }
    },
```

6. 新加入网络的容器间相互访问

```
nick@ubuntu20:~$ docker exec -it alpine1 ash
/ # ping alpine2
PING alpine2 (172.19.0.5): 56 data bytes
64 bytes from 172.19.0.5: seq=0 ttl=64 time=0.116 ms
64 bytes from 172.19.0.5: seq=1 ttl=64 time=0.152 ms
^C
--- alpine2 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.116/0.134/0.152 ms
/ # ping alpine3
PING alpine3 (172.19.0.2): 56 data bytes
64 bytes from 172.19.0.2: seq=0 ttl=64 time=0.104 ms
64 bytes from 172.19.0.2: seq=1 ttl=64 time=0.130 ms
^C
--- alpine3 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.104/0.117/0.130 ms
/ # ping alpine4
PING alpine4 (172.19.0.3): 56 data bytes
64 bytes from 172.19.0.3: seq=0 ttl=64 time=0.352 ms
64 bytes from 172.19.0.3: seq=1 ttl=64 time=0.138 ms
^C
--- alpine4 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.138/0.245/0.352 ms
/ #
```

7. 从自定义网络中移除所有容器，加入默认网络中的容器无法再不停服的情况下与docker0网桥断开

```
docker network disconnect alpine-net alpine1
docker network disconnect alpine-net alpine2
docker network disconnect alpine-net alpine3
docker network disconnect alpine-net alpine4
```

```
nick@ubuntu20:~$ docker network inspect alpine-net
[
    {
        "Name": "alpine-net",
        "Id": "957175b97b06d11b40a2b62ea5bdbf57166da9da8e4edbd8f924520afae74b79",
        "Created": "2023-07-27T09:56:56.175354509Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.19.0.0/16",
                    "Gateway": "172.19.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {},
        "Options": {},
        "Labels": {}
    }
]
```

8. 移除网络

```
docker network rm alpine-net
```

```
nick@ubuntu20:~$ docker network rm alpine-net
alpine-net
nick@ubuntu20:~$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
6f029e56a44b   bridge    bridge    local
dac84b5c3a28   host      host      local
5973d31883da   none      null      local
nick@ubuntu20:~$
```

## 自定义bridge网络2

1. 自定义网络，指定网关，子网，ip范围以及特定选项

```
docker network create \
--driver=bridge \
--subnet=172.28.0.0/16 \
--ip-range=172.28.5.0/24 \
--gateway=172.28.5.254 \
--opt com.docker.network.bridge.name=alpine-net1 \
--opt com.docker.network.bridge.enable_ip_masquerade=false \
--opt com.docker.network.bridge.enable_icc=false \
--opt com.docker.network.container_iface_prefix=ethnick \
alpine-net1
```

```
nick@master1:~$ docker network create \
> --driver=bridge \
> --subnet=172.28.0.0/16 \
> --ip-range=172.28.5.0/24 \
> --gateway=172.28.5.254 \
> --opt com.docker.network.bridge.name=alpine-net1 \
> --opt com.docker.network.bridge.enable_ip_masquerade=false \
> --opt com.docker.network.bridge.enable_icc=false \
> --opt com.docker.network.container_iface_prefix=ethnick \
> alpine-net1
e47461f6a0bb3c060ce7a89e6db2ac8f1331aaf043275314ee99d2384199f7d0
```

```
nick@master1:~$ ifconfig
alpine-net1: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.28.5.254  netmask 255.255.0.0  broadcast 172.28.255.255
        ether 02:42:d0:30:54:68  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

docker0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        inet6 fe80::42:84ff:feed:9211  prefixlen 64  scopeid 0x20<link>
        ether 02:42:84:ed:92:11  txqueuelen 0  (Ethernet)
        RX packets 5226  bytes 242093 (242.0 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 5919  bytes 29350761 (29.3 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

2. 查看网络详情

```
nick@master1:~$ docker network inspect alpine-net1
[
    {
        "Name": "alpine-net1",
        "Id": "e47461f6a0bb3c060ce7a89e6db2ac8f1331aaf043275314ee99d2384199f7d0",
        "Created": "2023-07-29T09:25:48.295608965Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.28.0.0/16",
                    "IPRange": "172.28.5.0/24",
                    "Gateway": "172.28.5.254"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {
            "5be3a5ea2c01026ea5306e2b508cfea47c4f8d0f02461281b039d7ea3a75dea1": {
                "Name": "alpine6",
                "EndpointID": "4321e3e9c7534ede4bffe3f2b7d48110e97b03e9db99be5ba23f5914dd60a86a",
                "MacAddress": "02:42:ac:1c:05:00",
                "IPv4Address": "172.28.5.0/16",
                "IPv6Address": ""
            },
            "807fbe63f52b2b82a269d1aa2cecdce92179ed5f684e4efb1feebb97a17b8a44": {
                "Name": "alpine7",
                "EndpointID": "623a542251d959365d1b7ae73da7f97a09642bb6ed451730e5a7f2d53bb1cadb",
                "MacAddress": "02:42:ac:1c:05:01",
                "IPv4Address": "172.28.5.1/16",
                "IPv6Address": ""
            }
        },
        "Options": {
            "com.docker.network.bridge.enable_icc": "false",
            "com.docker.network.bridge.enable_ip_masquerade": "false",
            "com.docker.network.bridge.name": "alpine-net1",
            "com.docker.network.container_iface_prefix": "ethnick"
        },
        "Labels": {}
    }
]
```

3. 运行容器

```
docker run -dit --rm --name alpine6 --network alpine-net1 myalpine ash
docker run -dit --rm --name alpine7 --network alpine-net1 myalpine ash
```

4. 容器间无法相互访问（com.docker.network.bridge.enable_icc=false），但并不是不能dns解析

```
nick@master1:~$ docker exec -it alpine6 ash
/ # ping alpine7
PING alpine7 (172.28.5.1): 56 data bytes
```

容器间无法相互访问

```
/ # ping alpine7
PING alpine7 (172.28.5.1): 56 data bytes
^C
--- alpine7 ping statistics ---
109 packets transmitted, 0 packets received, 100% packet loss
/ # ping 172.28.5.1
PING 172.28.5.1 (172.28.5.1): 56 data bytes
```

ip地址也无法访问

5. 禁用了ip伪装无法访问外网（com.docker.network.bridge.enable_ip_masquerade=false）

```
/ # ping alpine7
PING alpine7 (172.28.5.1): 56 data bytes
^C
--- alpine7 ping statistics ---
109 packets transmitted, 0 packets received, 100% packet loss
/ # ping 172.28.5.1
PING 172.28.5.1 (172.28.5.1): 56 data bytes
^C                                                    无法访问外网
--- 172.28.5.1 ping statistics ---
84 packets transmitted, 0 packets received, 100% packet loss
/ # ping www.0voice.com
PING www.0voice.com (42.194.128.13): 56 data bytes
```

6. 网络接口前缀为ethnick（com.docker.network.container_iface_prefix=ethnick）

```
/ # ifconfig
ethnick0  Link encap:Ethernet  HWaddr 02:42:AC:1C:05:00
          inet addr:172.28.5.0  Bcast:172.28.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:49 errors:0 dropped:0 overruns:0 frame:0
          TX packets:256 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:5787 (5.6 KiB)  TX bytes:24584 (24.0 KiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:8 errors:0 dropped:0 overruns:0 frame:0
          TX packets:8 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:491 (491.0 B)  TX bytes:491 (491.0 B)
```

# Overlay（覆盖网络）

## 什么是overlay网络

1. overlay网络驱动程序在多个Docker守护进程主机之间创建分布式网络。该网络位于（覆盖）特定于主机的网络之上，允许连接到该网络的容器（包括群集服务容器）在启用加密时安全通信。Docker透明地处理每个数据包往返于正确的Docker守护进程主机和正确的目标容器的路由
2. 初始化一个swarm集群或将Docker主机加入现有swarm集群时，会在该Docker主机上创建两个新网络：
    1. 名为ingress的overlay网络，处理与swarm service相关的控制和数据流量。创建一个service没有指定自定义网络时，默认将连接到ingress网络。ingress网络提供对容器化应用程序的负载均衡和路由功能。它允许外部流量通过单一入口点访问多个容器，并根据定义的规则将请求转发到适当的后端容器
    2. 名为docker-gwbridge的bridge网络，用于将覆盖网络（包括ingress网络）连接到单个Docker守护程序的物理网络。通过这个网络，容器可以连接到宿主机
3. 可以使用docker network create创建自定义的overlay网络，方法与创建自定义bridge网络相同。服务或容器一次可以连接到多个网络。服务或容器只能通过各自连接的网络进行通信。

## overlay驱动使用案例

### 使用默认的overlay网络

1. 初始化swarm集群，会默认创建ingress 网络和docker_gwbridge网络

```
nick@k8s-master1:~$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
05fc5ec6b399   bridge    bridge    local
b67fd34d6012   host      host      local
16720bdf5232   none      null      local
nick@k8s-master1:~$ docker swarm init
Swarm initialized: current node (vayqlni1zm5a864gxx9v5sqer) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-5wjwwk1rmuxcwf2ioqhyhr8frkia4vormqtqpppygqt0h211zl-2o08xjxongw31qdq4aofomkxg 192.168.239.142:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

nick@k8s-master1:~$ docker network ls
NETWORK ID     NAME              DRIVER    SCOPE
05fc5ec6b399   bridge            bridge    local
fd2b25be39b6   docker_gwbridge   bridge    local
b67fd34d6012   host              host      local
nfa2xjix6wcr   ingress           overlay   swarm
16720bdf5232   none              null      local
nick@k8s-master1:~$
```

2. 添加两个worker节点到集群

```
# 获取加入集群的命令
docker swarm join-token worker
```

```
nick@master1:~$ docker node ls
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
pyvol2kux963quhyj4013u0kv *   master1    Ready     Active         Leader           24.0.5
y7u7qs1i58qw11dagwlag4fc1     worker1    Ready     Active                          24.0.5
4ycqztnowfonvkpckiwdnjvl3     worker2    Ready     Active                          24.0.5
nick@master1:~$
```

3. 创建service，将加入默认ingress网络

```
docker service create -p 8080:80 --replicas 3 --name nginx-svc nginx:latest
```

```
nick@master1:~$ docker service ls
ID             NAME        MODE         REPLICAS   IMAGE          PORTS
vrvda9egwytz   nginx-svc   replicated   3/3        nginx:1.23.4   *:8080->80/tcp
```

4. 查看ingress 网络明细

```
docker network inspect ingress
```

```
nick@master1:~$ docker network inspect ingress
[
    {
        "Name": "ingress",
        "Id": "nfa2xjix6wcrval5p4fu63ao6",
        "Created": "2023-07-27T13:32:00.507600089Z",
        "Scope": "swarm",
        "Driver": "overlay",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "10.0.0.0/24",
                    "Gateway": "10.0.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": true,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {
            "ad64d248fdee73b1e1e6804d25573e116576d591c6243a588e9fc6cacbeb4f69": {
                "Name": "nginx-svc.1.plvklgxvob2pg822wd7xdnmb1",
                "EndpointID": "08638b546348f8b0ba219c77c1e92ecec45a687a18673d8a24f6eb5dcc3a83b4",
                "MacAddress": "02:42:0a:00:00:0a",
                "IPv4Address": "10.0.0.10/24",
                "IPv6Address": ""
            },
            "ingress-sbox": {
                "Name": "ingress-endpoint",
                "EndpointID": "ea4c63d553eae09eb1b0f21c0c61481dbb0177cc93ee578c1e0c4f29f5e6619c",
                "MacAddress": "02:42:0a:00:00:02",
                "IPv4Address": "10.0.0.2/24",
                "IPv6Address": ""
            }
        },
        "Options": {
            "com.docker.network.driver.overlay.vxlanid_list": "4096"
        },
        "Labels": {},
        "Peers": [
            {
                "Name": "030bc2340de7",
                "IP": "192.168.239.142"
            },
            {
                "Name": "8cddb0f8815f",
                "IP": "192.168.239.143"
            },
            {
                "Name": "9d8609c3ee70",
                "IP": "192.168.239.144"
            }
        ]
    }
]
```

5. 查看docker_gwbridge网络详情

```
docker network inspect docker_gwbridge
```

```
nick@master1:~$ docker network inspect docker_gwbridge
[
    {
        "Name": "docker_gwbridge",
        "Id": "fd2b25be39b6d9e0786ac49c7afbbc9e5ecfe1cb37ac450e19d89f6ee3f66b26",
        "Created": "2023-07-27T13:32:01.37994085Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "172.18.0.0/16",
                    "Gateway": "172.18.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {
            "ad64d248fdee73b1e1e6804d25573e116576d591c6243a588e9fc6cacbeb4f69": {
                "Name": "gateway_a2e93a86c2bf",
                "EndpointID": "3795e94018820556c06b0895f62f8a88ad951cf5a4fd94dbf1997530df74b579",
                "MacAddress": "02:42:ac:12:00:03",
                "IPv4Address": "172.18.0.3/16",
                "IPv6Address": ""
            },
            "ingress-sbox": {
                "Name": "gateway_ingress-sbox",
                "EndpointID": "55a792ba133cfcf160a3708e2aeeac9d0bf965e9279d15fc807b6c75822dcef4",
                "MacAddress": "02:42:ac:12:00:02",
                "IPv4Address": "172.18.0.2/16",
                "IPv6Address": ""
            }
        },
        "Options": {
            "com.docker.network.bridge.enable_icc": "false",
            "com.docker.network.bridge.enable_ip_masquerade": "true",
            "com.docker.network.bridge.name": "docker_gwbridge"
        },
        "Labels": {}
    }
]
```

7. 查看iptables网络转发

```
nick@master1:~$ sudo iptables -nvL -t nat
Chain PREROUTING (policy ACCEPT 2437 packets, 146K bytes)
 pkts bytes target     prot opt in     out     source               destination
32400 1944K DOCKER-INGRESS  all  --  *      *       0.0.0.0/0            0.0.0.0/0            ADDRTYPE match dst-type LOCAL
32402 1958K DOCKER     all  --  *      *       0.0.0.0/0            0.0.0.0/0            ADDRTYPE match dst-type LOCAL

Chain INPUT (policy ACCEPT 2437 packets, 146K bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 98 packets, 6988 bytes)
 pkts bytes target     prot opt in     out     source               destination
  170 12567 DOCKER-INGRESS  all  --  *      *       0.0.0.0/0            0.0.0.0/0            ADDRTYPE match dst-type LOCAL
    1    60 DOCKER     all  --  *      *       0.0.0.0/0           !127.0.0.0/8          ADDRTYPE match dst-type LOCAL

Chain POSTROUTING (policy ACCEPT 97 packets, 6725 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 MASQUERADE  all  --  *      !docker_gwbridge  172.18.0.0/16        0.0.0.0/0
   23  5271 MASQUERADE  all  --  *      docker_gwbridge  0.0.0.0/0            0.0.0.0/0            ADDRTYPE match src-type LOCAL
  104  7026 MASQUERADE  all  --  *      !docker0  172.17.0.0/16        0.0.0.0/0

Chain DOCKER (2 references)
 pkts bytes target     prot opt in     out     source               destination
    0     0 RETURN     all  --  docker_gwbridge *       0.0.0.0/0            0.0.0.0/0
   11   660 RETURN     all  --  docker0 *       0.0.0.0/0            0.0.0.0/0

Chain DOCKER-INGRESS (2 references)
 pkts bytes target     prot opt in     out     source               destination
    0     0 DNAT       tcp  --  *      *       0.0.0.0/0            0.0.0.0/0            tcp dpt:8080 to:172.18.0.2:8080
32567 1957K RETURN     all  --  *      *       0.0.0.0/0            0.0.0.0/0
nick@master1:~$
```

所有发送到本机8080端口的数据被转发到172.18.0.2:8080，172.18.0.2 为docker_gwbridge 网络上ingress_sbox容器的地址，ingress_sbox实为网络命名空间并非容器，通过ingress_sbox网络命名空间连接ingress与docker_gwbridge两个设备。宿主机网络命名空间所在目录：/var/run/docker/netns

```
nick@master1:~$ sudo ls /var/run/docker/netns
172c0b1b8236  1-nfa2xjix6w  c9ec901662fb  ingress_sbox
```

8. 通过nsenter命令在ingress_sbox命名空间下运行ash 终端（nsenter是一个可以在指定进程的命令空间下运行指定程序的命令）

```
# 已超级权限启动容器，并在容器中以指定的命名空间运行程序
docker run -dit --rm -v /var/run/docker/netns:/netns --privileged=true \
--name alpine5 myalpine nsenter --net=/netns/ingress_sbox ash
# 与容器交互查看容器中网卡信息ifconfig
docker exec -it alpine5 ash
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
7: eth0@if8: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1450 qdisc noqueue state UP
    link/ether 02:42:0a:00:00:02 brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.2/24 brd 10.0.0.255 scope global eth0
       valid_lft forever preferred_lft forever
    inet 10.0.0.9/32 scope global eth0
       valid_lft forever preferred_lft forever
9: eth1@if10: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue state UP
    link/ether 02:42:ac:12:00:02 brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.2/16 brd 172.18.255.255 scope global eth1
       valid_lft forever preferred_lft forever
```

ingress网络

service 服务的虚拟ip

docker_gwbridge网络

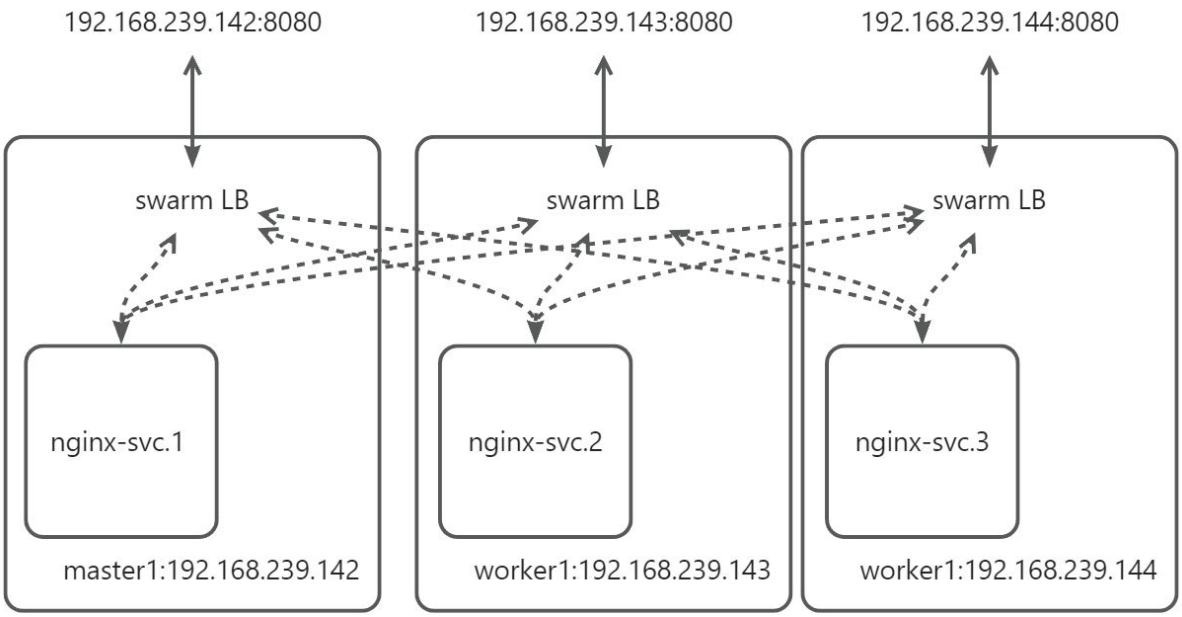
ingress_sbox网络命名空间连接了ingress网络与docker_gwbridge网络

9. 通过ipvsadm查看虚拟ip

```
/ # ipvsadm
IP Virtual Server version 1.2.1 (size=4096)
Prot LocalAddress:Port Scheduler Flags
  -> RemoteAddress:Port           Forward Weight ActiveConn InActConn
FWM  258 rr
  -> 10.0.0.15:0                  Masq    1      0          0
  -> 10.0.0.16:0                  Masq    1      0          0
  -> 10.0.0.17:0                  Masq    1      0          0
```

为服务下容器的虚拟ip，通过虚拟ip实现负载均

10. 示意图



## 自定义默认的ingress网络与docker_gwbridge网络

### 自定义默认的ingress（manager节点）

1. 移除原有默认ingress

```
docker network rm  ingress
```

```
nick@master1:~$ docker network rm  ingress
WARNING! Before removing the routing-mesh network, make sure all the nodes in your swarm run the same docker engine version. Otherwise, removal may not be effective and functio
lity of newly create ingress networks will be impaired.
Are you sure you want to continue? [y/N] y
ingress
nick@master1:~$
```

警告！在移除路由网状网络之前，请确保群中的所有节点都运行相同的docker引擎版本。否则，移除可能无效，新创建的入口网络的功能将受损

2. 使用--ingress 标志创建overlay网络

```
docker network create \
  --driver overlay \
  --ingress \
  --subnet=10.10.0.0/16 \
  --gateway=10.10.0.2 \
  --opt com.docker.network.driver.mtu=1500 \
  my-ingress
```

### 自定义默认docker_gwbridge（worker节点）

1. 自定义docker_gwbridge必须使节点脱离集群

```
docker swarm leave
```

2. 移除原有docker_gwbridge网络

```
docker network rm docker_gwbridge
```

3. 创建新的docker_gwbridge网络

```
docker network create \
--subnet 10.11.0.0/16 \
--opt com.docker.network.bridge.name=docker_gwbridge \
--opt com.docker.network.bridge.enable_icc=false \
--opt com.docker.network.bridge.enable_ip_masquerade=true \
docker_gwbridge
```

4. 重新将节点加入集群

### 验证自定义网络

1. 使用默认网络创建服务验证效果

```
docker service create -p 8080:80 --replicas 3 --name nginx-svc nginx:latest
```

2. 修改了docker_gwbridge网络的worker节点上运行工具容器

```
# 已超级权限启动容器，并在容器中以指定的命名空间运行程序
docker run -dit --rm -v /var/run/docker/netns:/netns --privileged=true \
--name alpine5 myalpine nsenter --net=/netns/ingress_sbox ash
# 与容器交互查看容器中网卡信息ifconfig
docker exec -it alpine5 ash
```

```
/ # ipvsadm
IP Virtual Server version 1.2.1 (size=4096)
Prot LocalAddress:Port Scheduler Flags
  -> RemoteAddress:Port           Forward Weight ActiveConn InActConn
FWM  257 rr
  -> 10.10.0.6:0                  Masq    1      0          0
  -> 10.10.0.7:0                  Masq    1      0          0
  -> 10.10.0.8:0                  Masq    1      0          0
/ # exit
/ # ifconfig
eth0      Link encap:Ethernet  HWaddr 02:42:0A:0A:00:04
          inet addr:10.10.0.4  Bcast:10.10.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:1450  Metric:1
          RX packets:1 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:54 (54.0 B)  TX bytes:0 (0.0 B)

eth1      Link encap:Ethernet  HWaddr 02:42:0A:0B:00:02
          inet addr:10.11.0.2  Bcast:10.11.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:18 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:1532 (1.4 KiB)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

容器ip地址使用了新的IP网段

ingress与docker_gwbridge
均使用了配置网段

## 自定义overlay网络的使用

1. 创建自定义overlay网络

```
docker network create -d overlay \
--attachable \
--subnet=10.12.0.0/16 \
--subnet=10.13.0.0/16 \
--gateway=10.12.0.100 \
--gateway=10.13.0.100 \
nginx-net
```

2. 查看网络详情

```
nick@master1:~$ docker network inspect nginx-net
[
    {
        "Name": "nginx-net",
        "Id": "x1wga8b3gfof9zlix67r0hzp2",
        "Created": "2023-07-29T09:53:31.473185226Z",
        "Scope": "swarm",
        "Driver": "overlay",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "10.12.0.0/16",
                    "Gateway": "10.12.0.100"
                },
                {
                    "Subnet": "10.13.0.0/16",
                    "Gateway": "10.13.0.100"
                }
            ]
        },
        "Internal": false,
        "Attachable": true,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": null,
        "Options": {
            "com.docker.network.driver.overlay.vxlanid_list": "4100,4101"
        },
        "Labels": null
    }
]
```

3. 指定网络创建服务

```
docker service create -p 8081:80 --replicas 3 --name nginx-svc1 --network nginx-net nginx:latest
```

4. 查看服务详情

```
docker service inspect nginx-svc1
```

```
nick@master1:~$ docker service inspect nginx-svc1
[
    {
        "ID": "twsu0kimkzlemagemdlwzied6",
        "Version": {
            "Index": 149
        },
        "CreatedAt": "2023-07-31T02:53:10.080304194Z",
        "UpdatedAt": "2023-07-31T02:53:10.082989493Z",
        "Spec": {
            "Name": "nginx-svc1",
            "Labels": {},
            "TaskTemplate": {
                "ContainerSpec": {
                    "Image": "nginx:latest@sha256:67f9a4f10d147a6e04629340e6493c9703300ca23a2f7f3aa56fe615d75d31ca",
                    "Init": false,
                    "StopGracePeriod": 10000000000,
                    "DNSConfig": {},
                    "Isolation": "default"
                },
```

……

```
                },
                "Networks": [
                    {
                        "Target": "x1wga8b3gfof9zlix67r0hzp2"
                    }
                ],
                "ForceUpdate": 0,
                "Runtime": "container"
            },
            "Mode": {
                "Replicated": {
                    "Replicas": 3
                }
```

服务指定的网络ID

……

```
            ],
            "VirtualIPs": [
                {
                    "NetworkID": "lxw3zxo16kzb1qzjg97ugad5d",
                    "Addr": "10.10.0.8/16"
                },
                {
                    "NetworkID": "x1wga8b3gfof9zlix67r0hzp2",
                    "Addr": "10.12.0.11/16"
                }
            ]
```

服务于ingress网络中的集群IP

服务于nginx-net网络中的集群IP

ingress： 网络提供对容器化应用程序的负载均衡和路由功能。它允许外部流量通过单一入口点访问多个容器，并根据定义的规则将请求转发到适当的后端容器

nginx-net：自定义overlay网络，提供跨主机容器互联与跨主机通信

5. 每个服务中的任务（容器）均连接到ingress网络（提供负载与路由）与自定义overlay网络（跨节点通信）以及加入docker_gwbridge网络（容器与主机通信）

```
nick@master1:~$ docker inspect nginx-svc1.3.go8iaej8tj164f7yb0c9y3s4s
[
    {
        "Id": "abcfd5fc8a274f82bdc4555ea04d0b35764ad3977ab4c809eb13ef043d791302",
        "Created": "2023-07-31T02:53:10.943004359Z",
        "Path": "/docker-entrypoint.sh",
        "Args": [
            "nginx",
            "-g",
            "daemon off;"
        ],
        "State": {
            "Status": "running",
```

容器ID

```
        "NetworkSettings": {
            "Bridge": "",
            "SandboxID": "7709c52df9e400819a471129f3fdab9bfe1eb933267e805a368f63744c352aa3",
            "HairpinMode": false,
            "LinkLocalIPv6Address": "",
            "LinkLocalIPv6PrefixLen": 0,
            "Ports": {
                "80/tcp": null
            },
            "SandboxKey": "/var/run/docker/netns/7709c52df9e4",
            "SecondaryIPAddresses": null,
            "SecondaryIPv6Addresses": null,
            "EndpointID": "",
            "Gateway": "",
            "GlobalIPv6Address": "",
            "GlobalIPv6PrefixLen": 0,
            "IPAddress": "",
            "IPPrefixLen": 0,
            "IPv6Gateway": "",
            "MacAddress": "",
```

网络命名空间

```
            "my-ingress": {
                "IPAMConfig": {
                    "IPv4Address": "10.10.0.13"
                },
                "Links": null,
                "Aliases": [
                    "abcfd5fc8a27"
                ],
                "NetworkID": "lxw3zxo16kzb1qzjg97ugad5d",
                "EndpointID": "3e6a07529d3fd7a14cc01201d5124837a0beae7f76d1771159d7b62f0cc41870",
                "Gateway": "",
                "IPAddress": "10.10.0.13",
                "IPPrefixLen": 16,
                "IPv6Gateway": "",
                "GlobalIPv6Address": "",
                "GlobalIPv6PrefixLen": 0,
                "MacAddress": "02:42:0a:0a:00:0d",
                "DriverOpts": null
            },
```

加入的网络

```
            "nginx-net": {
                "IPAMConfig": {
                    "IPv4Address": "10.13.0.2"
                },
                "Links": null,
                "Aliases": [
                    "abcfd5fc8a27"
                ],
                "NetworkID": "x1wga8b3gfof9zlix67r0hzp2",
                "EndpointID": "ac11eae72fe089320f67f0aa65f0fb7e218e83ebb8e2dc75d83851a15daa39a3",
                "Gateway": "",
                "IPAddress": "10.13.0.2",
                "IPPrefixLen": 16,
                "IPv6Gateway": "",
                "GlobalIPv6Address": "",
                "GlobalIPv6PrefixLen": 0,
                "MacAddress": "02:42:0a:0d:00:02",
                "DriverOpts": null
            }
```

```
nick@master1:~$ docker network inspect docker_gwbridge
[
    {
        "Name": "docker_gwbridge",
        "Id": "b9ffb33f6b0e83df2ef13ce5fd490f1fa044df9c8b529139ab88d9285fec7f50",
        "Created": "2023-07-29T07:21:04.574418451Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "172.18.0.0/16",
                    "Gateway": "172.18.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {
            "abcfd5fc8a274f82bdc4555ea04d0b35764ad3977ab4c809eb13ef043d791302": {
                "Name": "gateway_7709c52df9e4",
                "EndpointID": "59a0392b3e45e034bbc8f8afcb9b8ca112b31abe802315ec8bd04719ea738caf",
                "MacAddress": "02:42:ac:12:00:03",
                "IPv4Address": "172.18.0.3/16",
                "IPv6Address": ""
            },
            "my-ingress-sbox": {
                "Name": "gateway_ingress_sbox",
                "EndpointID": "0317a8d3a04fddf5929cd23e73ff0b0bff1477c295a04f23996482c8a213c29d",
                "MacAddress": "02:42:ac:12:00:02",
```

通过网络命名空间与docker_gwbridge网络建立关系

6. 访问服务

```
curl http://192.168.239.142:8081
curl http://192.168.239.143:8081
curl http://192.168.239.144:8081
```

# Host（主机网络）

## 什么是Host网络

1. Host模式，即容器网络不会与宿主机产生网络隔离，而是使用主机的网络栈，容器不会分配自己的IP地址。运行容器是所有端口映射的选项都将失效，并且在使用时产生警告信息。主机模式可以用于性能优化，因为主机模式下无需网络地址转换（NAT），并且不会为每个端口创建userland-proxy（在默认情况下，Docker使用Userland Proxy作为默认的端口转发机制。Userland Proxy基于iptables规则进行转发，将容器内部的网络流量通过宿主机上的特定端口进行转发。）
2. Host模式，仅适用于linux主机，Mac和windows主机不支持
3. docker run 或docker service create 通过指定--network host 来使用主机网络。该模式下集群节点上运行容器将受到限制（例如：容器端口为80端口，由于使用主机网络，则每个集群节点仅能运行一个80端口的容器）

## 案例

1. 通过host 模式运行容器，无需指定端口映射

```
docker run -d --rm --name mynginx --network host  nginx:latest
curl http://localhost
```

```
nick@master1:~$ docker network inspect host
[
    {
        "Name": "host",
        "Id": "b67fd34d6012a2592abd8576aaaf1ca40c808bc3d2a6001b32e097114bc46509",
        "Created": "2022-11-29T07:48:49.934429539Z",
        "Scope": "local",
        "Driver": "host",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": []
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {
            "047b42ed574b96ebfb3c70f3e9e53cb2939b51f6661f8dd1f05c2a12d03900ca": {
                "Name": "mynginx",
                "EndpointID": "b1ae6c119c4fd68027e406aa86ed8e73f06df0c3bd691906e02b72be88242c87",
                "MacAddress": "",
                "IPv4Address": "",
                "IPv6Address": ""
            }
        },
        "Options": {},
        "Labels": {}
    }
```

host与主机共享网络，不具备独立的网络信息

2. 以host模式创建service，host 模式下使用主机的IP和端口，因此swarm不提负载均衡

```
docker service create --replicas 3 --name nginx-svc2 --network host nginx:latest
```

```
nick@master1:~$ docker service ps nginx-svc2
ID             NAME              IMAGE          NODE      DESIRED STATE   CURRENT STATE            ERROR
pv4pdzwc62p5   nginx-svc2.1      nginx:latest   worker1   Running         Running 57 seconds ago
54pa0ytriygu   nginx-svc2.2      nginx:latest   worker2   Running         Running 57 seconds ago
9rwdh2sjqr0i   nginx-svc2.3      nginx:latest   master1   Running         Running 25 seconds ago
bkydrrz4du9l    \_ nginx-svc2.3  nginx:latest   master1   Shutdown        Failed 31 seconds ago    "task: non-zero exit (1)"
rm66c2mxegy3    \_ nginx-svc2.3  nginx:latest   master1   Shutdown        Failed 39 seconds ago    "task: non-zero exit (1)"
bmp1135t7svf    \_ nginx-svc2.3  nginx:latest   master1   Shutdown        Failed 47 seconds ago    "task: non-zero exit (1)"
pd8l9m907cbj    \_ nginx-svc2.3  nginx:latest   master1   Shutdown        Failed 55 seconds ago    "task: non-zero exit (1)"
```

由于host 驱动共享主机网络，因此部署的服务在每个节点只能出现一个，否则将因为端口冲突导致容器启动失败

# IPvlan

## 什么是IPvlan

1. IPvlan驱动程序为用户提供了对IPv4和IPv6寻址的完全控制。VLAN驱动程序在此基础上构建，使运营商能够完全控制第2层VLAN标记，甚至为对底层网络集成感兴趣的用户提供IPvlan L3路由。
2. ipvlan网络驱动程序可以用于创建IPVLAN网络，它允许将容器连接到现有的物理网络接口，并通过共享主机内核栈来实现高性能和低延迟
3. 创建ipvlan驱动网络，--option选项
    1. ipvlan_mode：设置IPvlan模式默认值为 l2，可选值为：l2（容器和主机共享相同的MAC地址，但拥有不同的IP地址）、l3、l3s。
    2. ipvlan_flag：设置IPvlan模式flag，默认值为 bridge，可选值为：bridge，private，vepa（Virtual Ethernet Port Aggregator）
    3. parent：指定需要使用的父级网卡

# 内核要求

1. linux 内核版本 v4.2+
2. unname -r 查看内核版本

# 单机使用案例1（l2）

1. 通过ipvlan模式与宿主机共享物理接口

```
nick@master1:~$ route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0         192.168.239.2   0.0.0.0         UG    100    0        0 ens33
172.17.0.0      0.0.0.0         255.255.0.0     U     0      0        0 docker0
192.168.239.0   0.0.0.0         255.255.255.0   U     0      0        0 ens33
192.168.239.2   0.0.0.0         255.255.255.255 UH    100    0        0 ens33
nick@master1:~$
```

虚拟机默认网关为192.168.239.2

```
docker network create -d ipvlan \
--aux-address vmware=192.168.239.1 \
--subnet=192.168.239.0/24 --gateway=192.168.239.2 \
-o parent=ens33 -o ipvlan_mode=l2 \
pub_ens33
```

```
docker run -dit --rm --name alpine8 --network pub_ens33 myalpine ash
docker run -dit --rm --name alpine9 --network pub_ens33 myalpine ash
docker network inspect pub_ens33
```

```
    "ConfigOnly": false,
    "Containers": {
        "5bb6a83f202df1f61a714adaa62012d6cd63ca03f43c340ffe21b2cf843c8125": {
            "Name": "alpine9",
            "EndpointID": "cc604c38394ff9034f0a03bcfc5ac08be1af961d2516b20c1c1b2b3bb33fbd03",
            "MacAddress": "",
            "IPv4Address": "192.168.239.4/24",
            "IPv6Address": ""
        },
        "d5dd00b85e34c8e5bff1de5af089a68e79e9a75d7990bcf764ae0deb742f5ae2": {
            "Name": "alpine8",
            "EndpointID": "1fad7d9c3c277983ad603fa952578de10000e0e406ec4531fcd70821fbefafef",
            "MacAddress": "",
            "IPv4Address": "192.168.239.3/24",
            "IPv6Address": ""
        }
    },
```

1 192.168.239.142 | 2 192.168.239.143 | 3 192.168.239.142 | 4 192.168.239.143

```
nick@worker1:~$ ping 192.168.239.3
PING 192.168.239.3 (192.168.239.3) 56(84) bytes of data.
64 bytes from 192.168.239.3: icmp_seq=1 ttl=64 time=1.61 ms
64 bytes from 192.168.239.3: icmp_seq=2 ttl=64 time=1.14 ms
```

其他主机可以通过IP地址访问容器

1 192.168.239.142 | 2 192.168.239.143 | 3 192.168.239.142 | 4 192.168.239.143

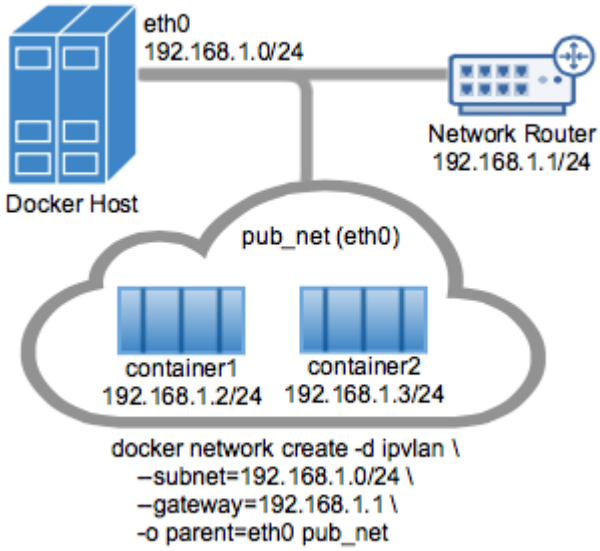
```
nick@master1:~$ docker exec -it alpine8 ash
/ # ping alpine9
PING alpine9 (192.168.239.4): 56 data bytes
64 bytes from 192.168.239.4: seq=0 ttl=64 time=2.121 ms
64 bytes from 192.168.239.4: seq=1 ttl=64 time=0.099 ms
64 bytes from 192.168.239.4: seq=2 ttl=64 time=0.098 ms
```

同一网络中容器之间可以相互访问

1 192.168.239.142 | 2 192.168.239.143 | 3 192.168.239.142 | 4 192.168.239.143

```
nick@master1:~$ ping 192.168.239.3
PING 192.168.239.3 (192.168.239.3) 56(84) bytes of data.
From 192.168.239.142 icmp_seq=1 Destination Host Unreachable
From 192.168.239.142 icmp_seq=2 Destination Host Unreachable
From 192.168.239.142 icmp_seq=3 Destination Host Unreachable
```

宿主机与容器之间无法相互访问

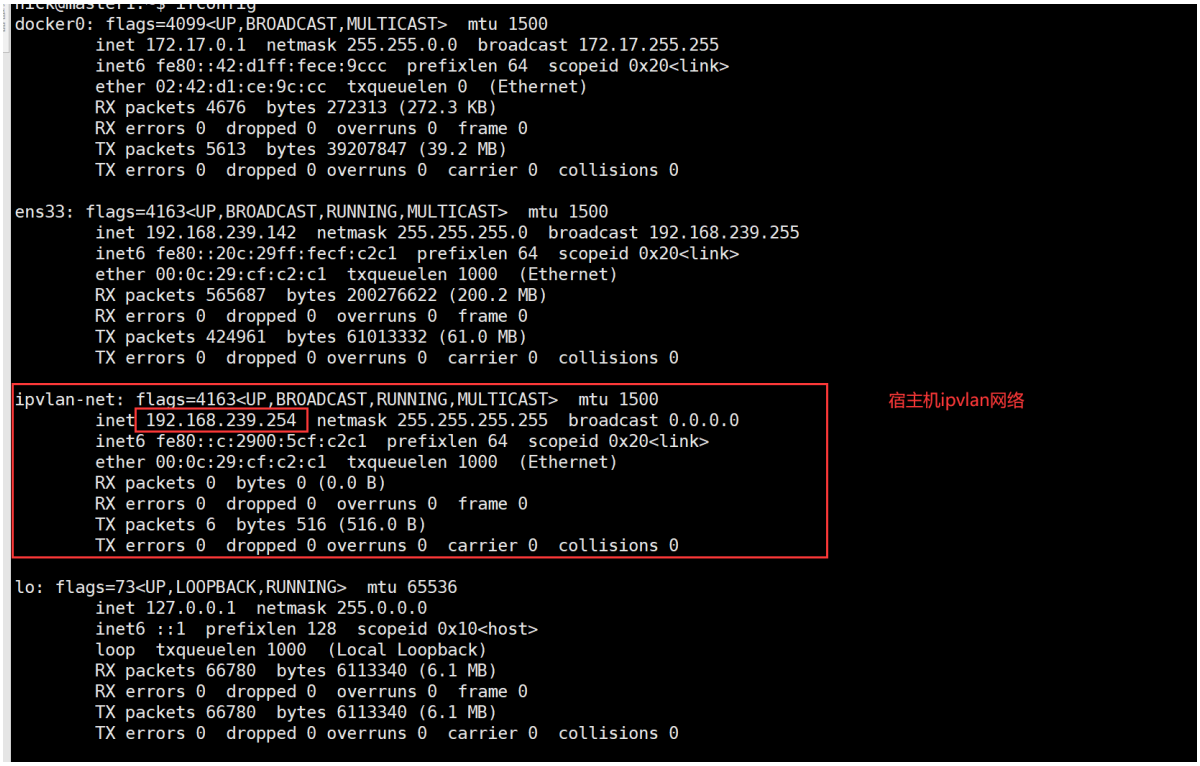


2. ipvlan与宿主机通讯

Docker 设计的时候就考虑了跨主机通讯和宿主机网络隔离的安全问题，在 VLAN 模式下只允许同 VLAN 的 MACVLAN/IPVLAN 子接口内部通讯，同时对来自宿主通过父接口到子接口的通讯进行了过滤(阻断)，以确保安全。因此可利用子接口可相互通讯的原理，在宿主多配置一个子接口，并多分配一个 IP 地址，同时在路由表配置正确的情况下，容器就能和物理网络完全互通了。

1. 宿主机创建ipvlan网络

```
# 宿主机创建ipvlan网络
# ip link add <网络名称> link <物理网卡> type ipvlan mode <mode：l2、l3、l3s>
sudo ip link add ipvlan-net link ens33 type ipvlan mode l2
# 为新建的ipvlan网络指定IP地址
sudo ip addr add 192.168.239.254 dev ipvlan-net
# 启动新建的网络
sudo ip link set ipvlan-net up
```




2. 启用网络接口混杂支持，网络接口混杂模式（Promiscuous Mode）是指网络接口卡在收发数据时，不仅会接收自己的目标MAC地址或广播地址的数据包，还会接收其他所有目标MAC地址的数据包。这种模式可以用于网络分析、嗅探和安全监测等应用

```
# 启用父接口混杂支持
sudo ip link set ens33 promisc on
# 启用子接口混杂支持
sudo ip link set ipvlan-net promisc on
# 查看网络信息
ip addr
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,PROMISC,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:cf:c2:c1 brd ff:ff:ff:ff:ff:ff
    inet 192.168.239.142/24 brd 192.168.239.255 scope global dynamic ens33
       valid_lft 1166sec preferred_lft 1166sec
    inet6 fe80::20c:29ff:fecf:c2c1/64 scope link
       valid_lft forever preferred_lft forever
5: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:d1:ce:9c:cc brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
    inet6 fe80::42:d1ff:fece:9ccc/64 scope link
       valid_lft forever preferred_lft forever
111: ipvlan-net@ens33: <BROADCAST,MULTICAST,PROMISC,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN group default qlen 1000
    link/ether 00:0c:29:cf:c2:c1 brd ff:ff:ff:ff:ff:ff
    inet 192.168.239.254/32 scope global ipvlan-net
       valid_lft forever preferred_lft forever
    inet6 fe80::c:2900:5cf:c2c1/64 scope link
       valid_lft forever preferred_lft forever
```

已启用混杂模式

3. 新建docker专用路由表，宿主需要安装iproute2依赖库，默认已安装

```
# 通过文本编辑器，以root权限打开路由表文件
# 添加一个新的路由表专用于docker
sudo vim /etc/iproute2/rt_tables
```

```
nick@master1:~$ cat /etc/iproute2/rt_tables
#
# reserved values
#
255     local
254     main
253     default
0       unspec
#
# local
#
#1      inr.ruhep
```

修改前rt_tables数据

```
#
# reserved values
#
255     local
254     main
253     default
0       unspec
100     docker
#
# local
#
#1      inr.ruhep
~
~
```

```
# 刷新docker路由表
sudo ip route flush table docker
```

4. 添加路由与规则

```
# 向名为docker的路由表中添加一条默认路由
sudo ip route add default via 192.168.239.254 table docker
# 为子接口配置对应的静态路由表
sudo ip route add 192.168.239.0/24 dev ipvlan-net src 192.168.239.254 metric 10
# 配置本地回环到容器的路由规则
sudo ip rule add from all to 192.168.239.0/24 table docker
```

5. 查看信息

```
# 查看docker路由表规则
sudo ip route show table docker
# 查看子接口配置的静态路由表
route -n
# 查看所有的规则
sudo ip rule show
# 查看指定table id的规则
sudo ip rule show table 100
```

```
nick@master1:~$ sudo ip route show table docker
default via 192.168.239.254 dev ipvlan-net
nick@master1:~$ route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0         192.168.239.2   0.0.0.0         UG    100    0        0 ens33
172.17.0.0      0.0.0.0         255.255.0.0     U     0      0        0 docker0
192.168.239.0   0.0.0.0         255.255.255.0   U     0      0        0 ens33
192.168.239.0   0.0.0.0         255.255.255.0   U     10     0        0 ipvlan-net
192.168.239.2   0.0.0.0         255.255.255.255 UH    100    0        0 ens33
nick@master1:~$
nick@master1:~$ sudo ip rule show
0:      from all lookup local
32765:  from all to 192.168.239.0/24 lookup docker
32766:  from all lookup main
32767:  from all lookup default
nick@master1:~$
nick@master1:~$ sudo ip rule show table docker
32765:  from all to 192.168.239.0/24 lookup docker
```

6. 宿主机尝试访问容器

```
ping 192.168.239.3
```

```
nick@master1:~$ ping 192.168.239.3
PING 192.168.239.3 (192.168.239.3) 56(84) bytes of data.
64 bytes from 192.168.239.3: icmp_seq=1 ttl=64 time=0.049 ms
64 bytes from 192.168.239.3: icmp_seq=2 ttl=64 time=0.078 ms
64 bytes from 192.168.239.3: icmp_seq=3 ttl=64 time=0.059 ms
64 bytes from 192.168.239.3: icmp_seq=4 ttl=64 time=0.055 ms
```

7. 还原主机设置

```
# 删除规则
sudo ip rule del from all to 192.168.239.0/24 table docker
sudo ip rule show table docker
# 删除静态路由
sudo ip route del 192.168.239.0/24 dev ipvlan-net src 192.168.239.254 metric 10
route -n
# 删除docker表中的路由
sudo ip route del default via 192.168.239.254 table docker
sudo ip route show table docker

# 通过文本编辑器，以root权限打开路由表文件
# 删除上文添加的路由表docker
sudo vim /etc/iproute2/rt_tables
# 刷新对应的路由表
sudo ip route flush table docker
```

```
# 关闭宿主机与子网络的混杂模式
# 关闭父接口混杂支持
sudo ip link set ens33 promisc off
# 关闭子接口混杂支持
sudo ip link set ipvlan-net promisc off
# 查看网络信息
ip addr

# 删除宿主机ipvlan网络接口
# 停用网卡
sudo ip link set ipvlan-net down
# 删除网卡
sudo ip link del dev ipvlan-net
```

## 跨主机容器通信（l2）

1. 创建ipvlan网络，分别在多台机器上创建ipvlan网络

```
docker network create -d ipvlan \
--aux-address vmware=192.168.239.1 \
--subnet=192.168.239.0/24 --gateway=192.168.239.2 \
-o parent=ens33 -o ipvlan_mode=l2 \
pub_ens33
```

2. 分别在机器上创建容器，并指定容器IP地址

```
# 机器1
docker run -dit --rm --name alpine10 --ip 192.168.239.10 --network pub_ens33
myalpine ash
docker run -dit --rm --name alpine11 --ip 192.168.239.11 --network pub_ens33
myalpine ash
# 机器2
docker run -dit --rm --name alpine12 --ip 192.168.239.12 --network pub_ens33
myalpine ash
docker run -dit --rm --name alpine13 --ip 192.168.239.13 --network pub_ens33
myalpine ash
```

```
nick@master1:~$ docker exec -it alpine10  ash
/ # ping 192.168.239.10
PING 192.168.239.10 (192.168.239.10): 56 data bytes
64 bytes from 192.168.239.10: seq=0 ttl=64 time=0.092 ms
64 bytes from 192.168.239.10: seq=1 ttl=64 time=0.084 ms
^C
--- 192.168.239.10 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.084/0.088/0.092 ms
/ # ping 192.168.239.11
PING 192.168.239.11 (192.168.239.11): 56 data bytes
64 bytes from 192.168.239.11: seq=0 ttl=64 time=0.106 ms
64 bytes from 192.168.239.11: seq=1 ttl=64 time=0.144 ms
^C
--- 192.168.239.11 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.106/0.125/0.144 ms
/ # ping 192.168.239.12
PING 192.168.239.12 (192.168.239.12): 56 data bytes
64 bytes from 192.168.239.12: seq=0 ttl=64 time=1.041 ms
64 bytes from 192.168.239.12: seq=1 ttl=64 time=1.553 ms
^C
--- 192.168.239.12 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 1.041/1.297/1.553 ms
/ # ping 192.168.239.13
PING 192.168.239.13 (192.168.239.13): 56 data bytes
64 bytes from 192.168.239.13: seq=0 ttl=64 time=0.931 ms
64 bytes from 192.168.239.13: seq=1 ttl=64 time=0.900 ms
^C
--- 192.168.239.13 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.900/0.915/0.931 ms
/ # ping www.0voice.com
PING www.0voice.com (42.194.128.13): 56 data bytes
64 bytes from 42.194.128.13: seq=0 ttl=128 time=30.625 ms
64 bytes from 42.194.128.13: seq=1 ttl=128 time=38.045 ms
```

访问同主机的容器

访问其他主机的容器

访问外网

## 单机跨网络通信（l3）

1. 创建多子网ipvlan网络

```
docker network create -d ipvlan \
--subnet=192.168.214.0/24 \
--subnet=10.1.214.0/24 \
-o parent=ens33 -o ipvlan_mode=l3 \
pub_ens33-1
```

2. 运行容器并指定网络与ip地址

```
docker run -dit --rm --name alpine14 --network pub_ens33-1 --ip=192.168.214.10
myalpine ash
docker run -dit --rm --name alpine15 --network pub_ens33-1 --ip=10.1.214.10
myalpine ash
```

3. 不同子网间容器通信

```
nick@nick:~$ docker exec -it alpine15 ash
/ # ping alpine14
PING alpine14 (192.168.214.10): 56 data bytes
64 bytes from 192.168.214.10: seq=0 ttl=64 time=0.850 ms
64 bytes from 192.168.214.10: seq=1 ttl=64 time=0.241 ms
64 bytes from 192.168.214.10: seq=2 ttl=64 time=0.202 ms
64 bytes from 192.168.214.10: seq=3 ttl=64 time=0.152 ms
64 bytes from 192.168.214.10: seq=4 ttl=64 time=0.173 ms
^C
--- alpine14 ping statistics ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max = 0.152/0.323/0.850 ms
/ #
```

4. ipvlan L3模式中外部网络默认情况下是不知道 ipvlan 虚拟出来的网络的，如果不在外部路由器上配置好对应的路由规则，ipvlan 的网络是不能被外部直接访问的

# Macvlan

## 什么是Macvlan

1. 一些应用程序，特别是遗留应用程序或监控网络流量的应用程序，期望直接连接到物理网络。在这种情况下，您可以使用macvlan网络驱动程序为每个容器的虚拟网络接口分配一个MAC地址，使其看起来像是直接连接到物理网络的物理网络接口。在这种情况下，您需要在Docker主机上指定一个物理接口用于Macvlan，以及网络的子网和网关。您甚至可以使用不同的物理网络接口隔离您的Macvlan网络
2. 注意一下几点：
    1. 由于IP地址耗尽或“VLAN扩散”（当您的网络中有大量不适当的唯一MAC地址时会出现这种情况），您可能会无意中降低网络性能。
    2. 您的网络设备需要能够处理“混杂模式”，即一个物理接口可以分配多个MAC地址。
    3. 如果您的应用程序可以使用bridge（在单个Docker主机上）或overlay（在多个Docker主机之间进行通信）来工作，那么从长远来看，这些解决方案可能会更好。
3. 创建macvlan驱动网络，--option选项
    1. macvlan_mode：默认为bridge，可选项有：bridge, vepa, passthru, private
    2. parent：指定需要使用的父级网卡

## 案例

1. 创建macvlan网络

```
docker network create -d macvlan \
--subnet=172.16.86.0/24 \
--gateway=172.16.86.1 \
-o parent=ens33 \
pub_mac_ens33
```

```
docker network create -d macvlan \
--aux-address vmware=192.168.239.1 \
--subnet=192.168.239.0/24 --gateway=192.168.239.2 \
-o parent=ens33 \
pub_mac_ens33
```

2. 创建容器并使用macvlan网络

```
docker run -dit --rm --name alpine16 --network pub_mac_ens33 myalpine ash
docker run -dit --rm --name alpine17 --network pub_mac_ens33 myalpine ash
```

3. 查看网络详情，为每个容器分配了独立的ip地址与mac地址

```
docker network inspect pub_mac_ens33
```

```
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "192.168.239.0/24",
                    "Gateway": "192.168.239.2",
                    "AuxiliaryAddresses": {
                        "vmware": "192.168.239.1"
                    }
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {
            "a83f82b24485b784ec90220d3b796099aca8ddd90adacf638331efd7e3acdad2": {
                "Name": "alpine17",
                "EndpointID": "25e519deba78e9465f61bb693a80d292fcd9e269ecbeadf50d83e510a21b902e",
                "MacAddress": "02:42:c0:a8:ef:04",
                "IPv4Address": "192.168.239.4/24",
                "IPv6Address": ""
            },
            "db665510867bd41a06249f512dcfb6b3c6dc0b09adccb1ba210537e498f8ed30": {
                "Name": "alpine16",
                "EndpointID": "7469f770f2af073e3b8d82496b517f7fdfe9550386dbefdcc4ccca39953077f3",
                "MacAddress": "02:42:c0:a8:ef:03",
                "IPv4Address": "192.168.239.3/24",
                "IPv6Address": ""
            }
        },
```


# None

1. 如果要完全隔离容器的网络堆栈，可以在启动容器时使用--network-none标志。在容器中，只创建环回设备